STRING MATCHING

Text edit pgms - To find all occurrences of a pattern in the text.

(*) The text is a document being edited, and the pattern searched for is a particular word given by the user.

Applications:
patterns in DNA sequences
web pages to queries.

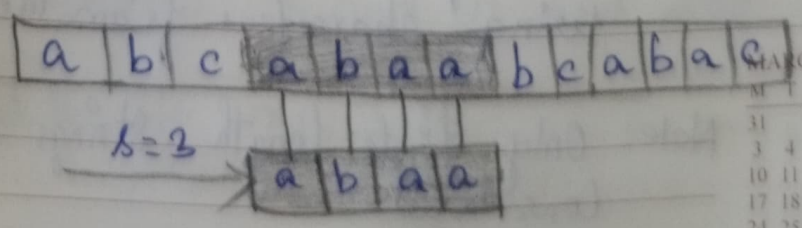
String matching prob:
The text is an array $T[1 \cdots n]$ of length n and the pattern is an array $P[1 \cdots m]$ of length $m \leq n$.

Assume: The elements of P and T are characters drawn from a finite alphabet Σ.

Example: $\Sigma = \{0, 1\}$ or $\Sigma = \{a, b, \ldots z\}$.

The character arrays P and T are often called strings of characters.

NOTES



MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

Pattern P occurs with shift s in text T
(or) pattern P occurs beginning at position $s+1$ in text T

if $0 \le s \le n-m$ and $T[s+1 \ldots s+m] = P[1 \ldots m]$.
(ie) if $T[s+j] = P[j]$, for $1 \le j \le m$.

* If P occurs with shift s in T, then we call s a valid shift; otherwise, we call s an invalid shift.

* The String-matching problem is the problem of finding all valid shifts with which a given pattern P occurs in a given text T.

String-matching algorithms:

Algorithm	Preprocessing time	Matching time
Naive	0	$O(n-m+1)m$
Rabin-Karp	$\Theta(m)$	$O(n-m+1)m$
finite automaton	$O(m\lvert\Sigma\rvert)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$

9 Sunday

Notation and Terminology:

Σ^* - set of all finite-length strings formed using characters from the alphabet Σ.

Note: Only finite length strings are considered.

ε - zero length empty string, also belongs to Σ^*.

$|x|$ - length of string x

xy - Concatenation of 2 string x and y of length $|x|+|y|$
consists of characters from x followed by the characters from y.

* A string w is a prefix of a string x, denoted $w \sqsubset x$, if $x = wy$ for some string $y \in \Sigma^*$.

Note: if $w \sqsubset x$, then $|w| \leq |x|$.

* A string w is a suffix of a string x, denoted $w \sqsupset x$, if $x = yw$ for some string $y \in \Sigma^*$.

Note: if $w \sqsupset x$, then $|w| \leq |x|$.

Example: $ab \sqsubset abcca$
$cca \sqsupset abcca$

Note: The empty string ε is both a suffix and prefix of every string.

* For any strings x and y and any character a,

$x \sqsupset y$ iff $xa \sqsupset ya$.

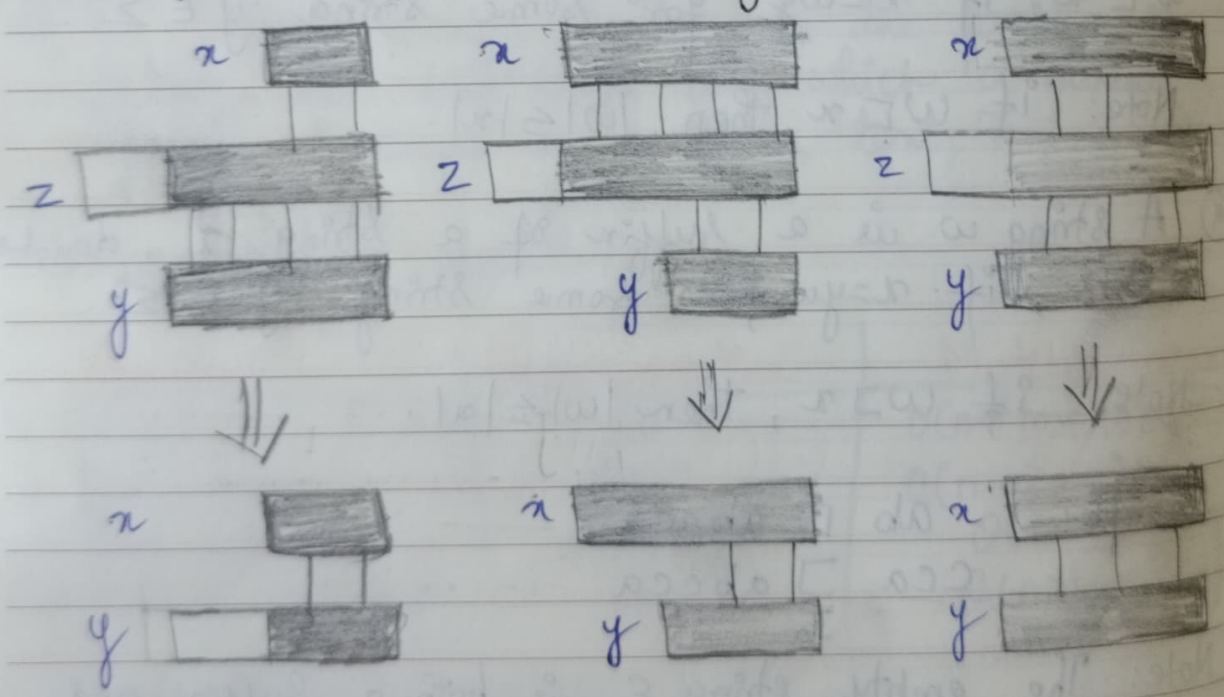
MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

Note: $\sqsubset$ and $\sqsupset$ are Transitive relations.

Overlapping-suffix lemma:

Let ~~If~~ x, y and z are strings such that $x \sqsupset z$ and $y \sqsupset z$.

If $|x| \leq |y|$ then $x \sqsupset y$.
If $|x| \geq |y|$ then $y \sqsupset x$.
If $|x| = |y|$ then $x = y$.



The naive string-matching algorithm:

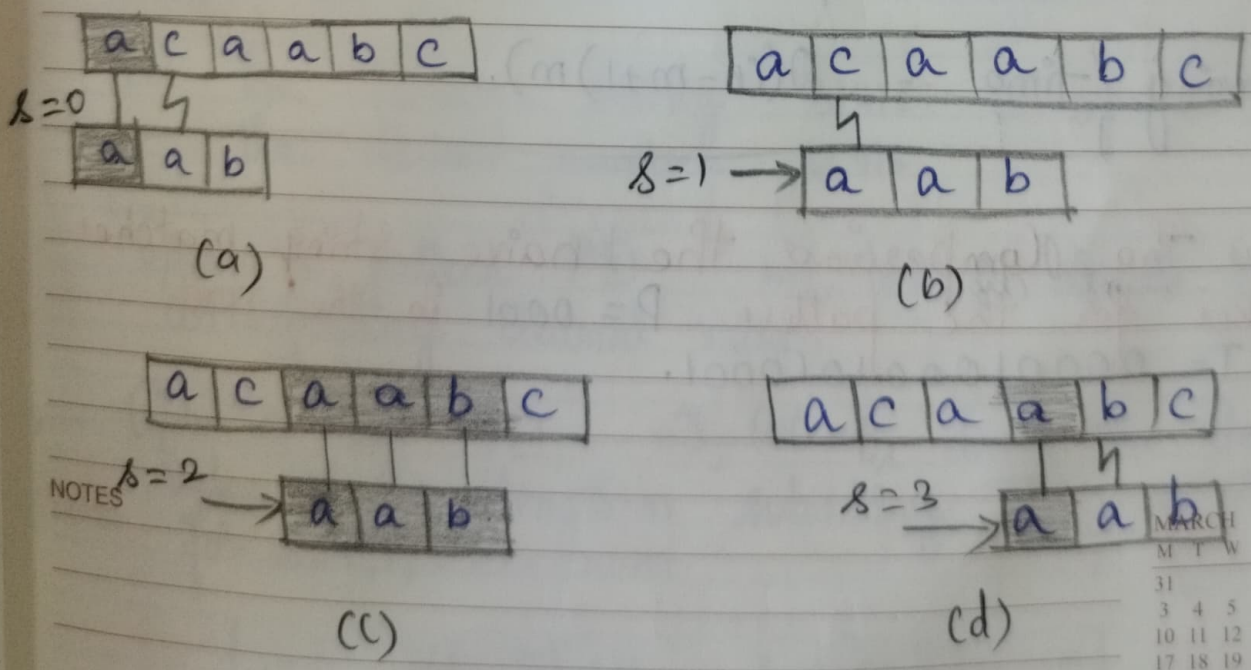
The naive algorithm finds all valid shifts using a loop that checks the condition

$P[1 \cdots m] = T[s+1 \cdots s+m]$ for each of the $n-m+1$ possible values of x.

NAIVE-STRING-MATCHER (T, P)

```
n = T.length
m = P.length
for s = 0 to n-m
    if P[1...m] == T[s+1 .... s+m]
        print "Pattern occurs with shift" s.
```

Ex: P = aab
T = acaabc



MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

* The procedure portrays as sliding a "template" containing the pattern over the text, noting for which shifts all of the characters on the template equal the corresponding characters in the text.

Lines 3-5: The for loop considers each possible shift explicitly.

Line 4 - test to determine whether the current shift is valid.

This test implicitly loops to check corresponding character positions until all positions match successfully or a mismatch is found.

Line 5 - prints out each valid shift s.

Running time $= O((n-m+1)m)$.

Show the comparisons the naive string matcher makes for the pattern $P = 0001$ in the text $T = 0000100010010001$.

LIFE INSURANCE AGENTS' FEDERATION OF INDIA
Regn. No. 2924 of 2000

The Rabin-Karp algorithm:

- Proposed by Rabin and Karp.
- Performs well in practice.
  Works for 2D pattern matching.
- Uses $\Theta(m)$ preprocessing time and $\Theta((n-m+1)m)$ running time.

Assume: $\Sigma = \{0, 1, 2, \ldots, 9\}$
each character is a decimal digit.
each character is a digit in <u>radix-d</u> notation, where $d = |\Sigma|$.

A string of K consecutive characters represents a length-K decimal number.

Ex: The character string 31415 corresponds to the decimal number 31,415.

Given a pattern $P[1 \ldots m]$, let p denote its corresponding decimal value.

Given a text $T[1 \ldots n]$, let t_s denote the decimal value of the length-m substring $T[s+1 \ldots s+m]$ for $s = 0, 1, \ldots, n-m$.

$$t_s = p, \text{ iff } T[s+1 \cdots s+m] = P[1 \cdots m]$$

NOTES

MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

$\Rightarrow$ s is a valid shift iff $t_s = p$.

Let p can be computed using Horner's rule:

$$p = P[m] + 10(p[m-1] + 10P[m-2] + \cdots + 10P[2] + 10P[1])$$

t_0 can be computed from $T[1 \cdots m]$.

$\rightarrow$ t_{s+1} can be computed from t_s as follows:

$$t_{s+1} = 10(t_s - 10^{m-1}T[s+1]) + T[s+m+1].$$

Subtracting $10^{m-1}T[s+1]$ removes high-order digit from t_s,

multiplying the result by 10 shifts the number left one digit position

adding $T[s+m+1]$ brings in the appropriate low-order digit.

Ex: If $m=5$ and $t_s = 31415$ then

remove the high-order digit $T[s+1] = 3$ and bring in new low-order digit $T[s+5+1] = 2$

$\therefore t_{s+1} = 10(31415 - 10000 \cdot 3) + 2$
$= 14152.$

* For larger values of p and t_s use a suitable modulus q.

* If q is chosen as prime then $10q$ just fits within one computer word, then all the necessary computations can be performed with single-precision arithmetic.

* In general, with a d-ary alphabet $\{0, 1, \ldots, d-1\}$, choose q so that dq fits within a computer word and the recurrence equation becomes.

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q.$$

where $h \equiv d^{m-1} \pmod q$ is the value of the digit "1" in the high-order position of an m-digit text window.

* Use the test $t_s \equiv p \pmod q$ as a fast heuristic test to rule out invalid shift s.

* Any shift s for which $t_s \equiv p \pmod q$ must be tested further to see whether is really valid or having a spurious hit.

This test additionally checks the condn $P[1 \ldots m] = T[s+1 \ldots s+m]$.

NOTES

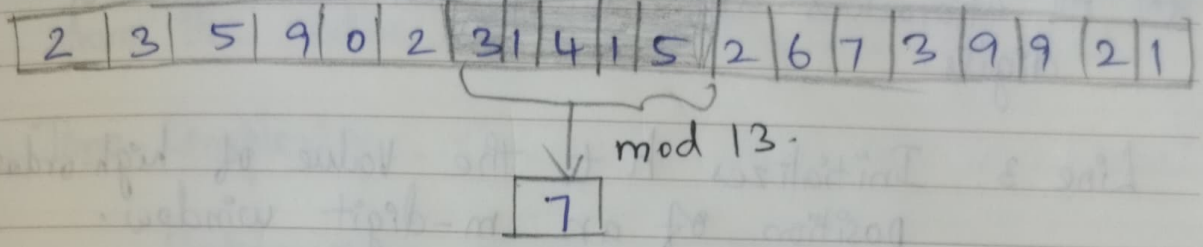
MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

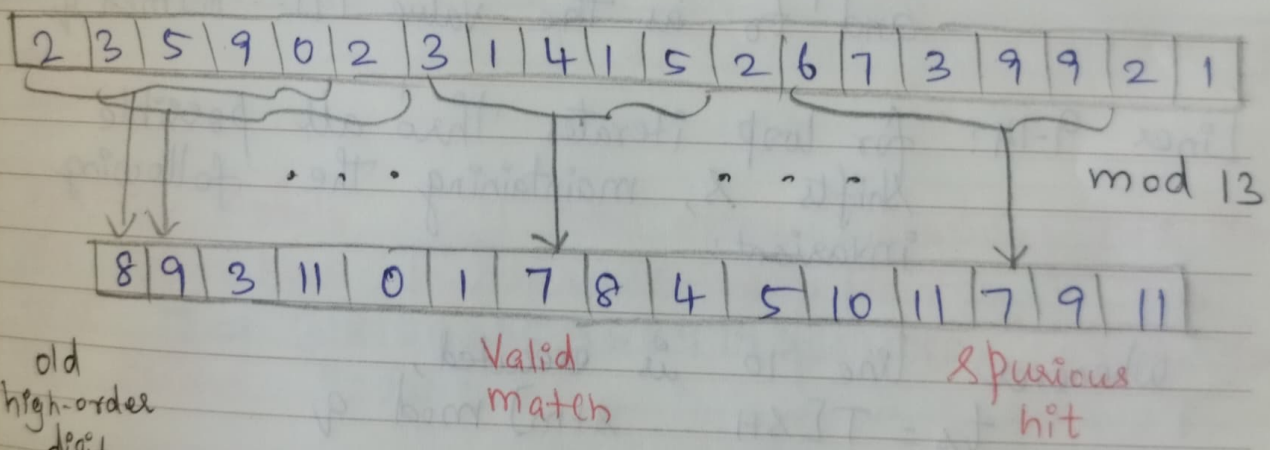
* If q is large enough, the spurious hits occur infrequently and the cost of extra checking is low.

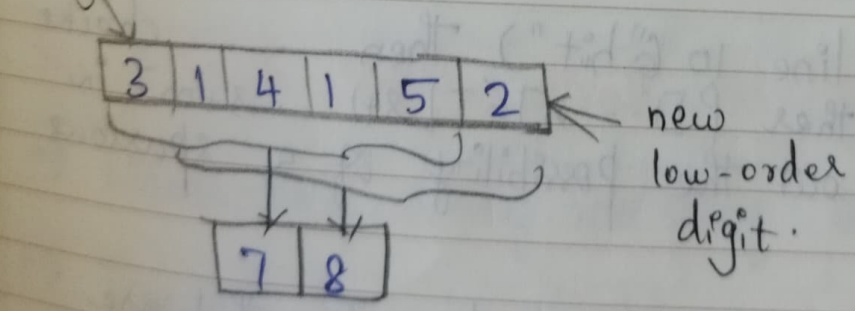
RABIN-KARP-MATCHER (T, P, d, q)

```
n = T.length
m = P.length
h = d^(m-1) mod q
p = 0
t0 = 0
for i = 1 to m                    // Preprocessing
    p = (dp + P[i]) mod q
    t0 = (dt0 + T[i]) mod q
for s = 0 to n-m                  // matching
    if p == ts
        if P[1...m] == T[s+1 ... s+m]
            print "Pattern occurs with shift" s
    if s < n-m
        ts+1 = (d(ts - T[s+1]h) + T[s+m+1]) mod q
```

* The inputs to the procedure are the text T, the pattern P, the radix d to use (usually $|\Sigma|$), and the prime q to use.







$$14152 = (31415 - 3 \times 10000) \times 10 + 2 \pmod{13}$$

$$= (7 - 3 \cdot 3) \cdot 10 + 2 \pmod{13}$$

$$= 8 \pmod{13}.$$

NOTES

* All characters are interpreted as radix-d digits.

Line 3: Initializes h to the value of high-order position of an m-digit window.

Lines 4-8: Compute p as the value $P[1 \ldots m] \bmod q$ and t_0 as the value $T[1 \ldots m] \bmod q$.

Lines 9-14: for loop iterates thro' all possible shifts s, maintaining the following invariant:

Whenever line 10 is executed,
$$t_s = T[s+1 \ldots s+m] \bmod q$$

* If $p = t_s$ in line 10 ("hit") then checks to see whether $P[1 \ldots n] = T[s+1 \ldots s+m]$ in order to rule out the possibility of a spurious hit.

Line 12 prints out any valid shifts that are found.

If $s < n-m$ (Checked in line 13), then the for loop will execute at least one more time, and so line 14 first executes to ensure that the loop invariant holds when gets back to line 10.

Line 14 computes the value of $t_{s+1} \bmod q$ from the value of $t_s \bmod q$.

Time-Complexity:

Preprocessing time $= \Theta(m)$
Matching time $= \Theta((n-m+1)m)$.

Problem:

Working modulo $q = 11$, how many spurious hits does the Rabin-Karp matcher ~~encoder~~ counter in the text $T = 31415926535897 93$ when looking for the pattern $P = 26$?

The Knuth-Morris-Pratt algorithm.

→ Linear-time string matching algorithm.

→ Invented by: Knuth, Morris and Pratt

→ Uses an auxiliary fn. π.

The prefix fn for a pattern: (π):

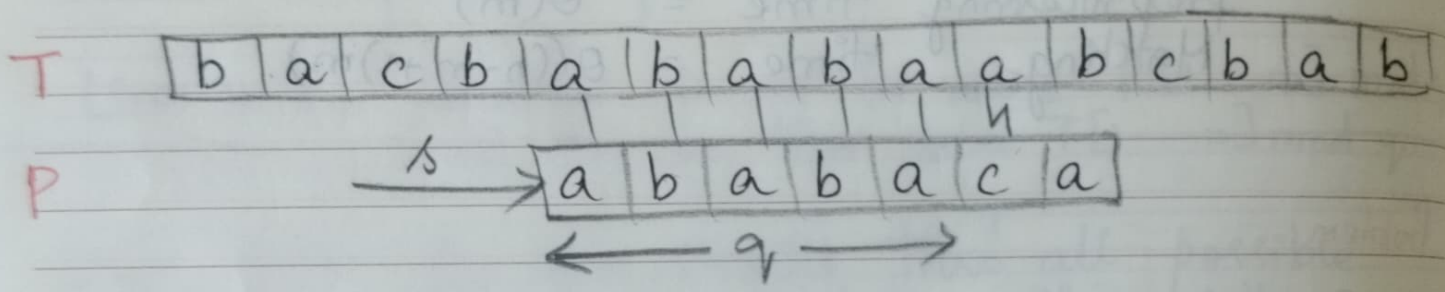
→ Encapsulates knowledge about how the pattern matches against shifts of itself.

NOTES

MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

→ Avoids testing useless shifts in naive pattern matching algo

Consider the operation of the naive string matcher:



The above figure shows a particular shift of s of a template containing the pattern P = ababaca against a text T.

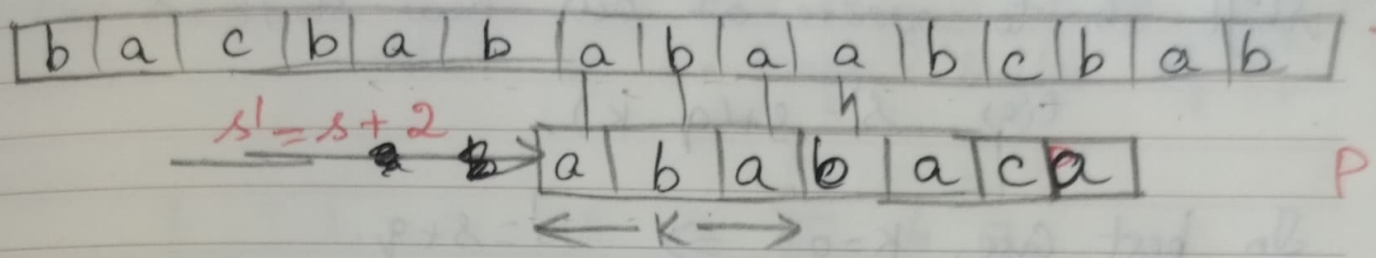
Here q=5 of the characters have matched successfully.
but the 6th pattern character fails to match the corresponding text character.

23 Sunday

→ Knowing these q characters allows us to determine immediately that certain shifts are invalid.

In the above figure,

NOTES Shift s+1 is necessarily invalid, first character a matches while the second character b does not match.



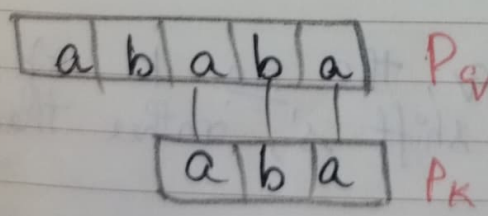
The shift $s' = s + 2$ aligns the first 3 pattern characters with 3 text characters that must necessarily match.

In general, it is useful to know

Given that pattern characters $P[1 \ldots q]$ match text characters $T[s+1 \ldots s+q]$, what is the least shift $s' > s$ such that for some $k < q$.

$$P[1 \ldots k] = T[s'+1 \ldots s'+k]$$

where. $s' + k = s + q$.



(ie) Knowing $P_q \sqsupset T_{s+q}$, we want the longest proper prefix P_k of P_q that is also a suffix of T_{s+q}.

The difference $q-k$ in the lengths of these prefixes of P is added to the

NOTES

MARCH 2025
M T W T F S S
31 1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

Shift s to arrive at our new shift s'

(ie) $s' = s + (q - k)$.

In best case, $k = 0 \Rightarrow s' = s + q$.

- At the new shift s', no need to compare the first k characters of P with the corresponding characters of T,
- The necessary information can be precomputed by comparing the pattern against itself.

Since $T[s'+1 \ldots s'+k]$ is part of the known portion of the text, it is a suffix of the string P_q.

The new shift $s' = s + (q - k)$ is the next potentially valid shift.

$\Rightarrow$ For each value of q, the no k of matching characters at the new shift s', rather than storing $s' - s$.

$\therefore$ Given a pattern $P[1 \ldots m]$, the prefix fn for the pattern P is the fn

NOTE: $\pi : \{1, 2, \ldots, m\} \rightarrow \{0, 1, \ldots m-1\}$

such that

$\pi[q] = \max\{k : k < q \text{ and } P_k \sqsupset P_q\}$.

FEBRUARY 2025
M T W T F S S
1 2
3 4 5 6 7 8 9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28

(ie) $\pi[q]$ – length of the longest prefix of P that is a proper suffix of P_q.

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

P_5 a b a b a c a

$\pi[5]=3$ P_3 a b a b a c a

$\pi[3]=1$ P_1 a b a b a c a

$\pi[1]=0$ P_0 a b a b a c a

KMP-MATCHER (T, P) // matches text T against P

```
n = T.length
m = P.length
π = COMPUTE-PREFIX-FUNCTION(P)
q = 0                          // no of characters matched
for i = 1 to n                 // scan the text from left to right
    while q > 0 and P[q+1] ≠ T[i]
        q = π[q]               // next character does not match
    if P[q+1] == T[i]
```

```
        q = q+1                          // next character matches
    if q == m                            // is all of P matched?
        print "Pattern occurs with shift" i-m
        q = π[q]                         // look for the next match
```

COMPUTE-PREFIX-FUNCTION (P) // matches P against itself

```
m = P.length
let π[1...m] be a new array
π[1] = 0
k = 0
for q = 2 to m
    while k>0 and P[k+1] ≠ P[q]
        k = π[k]
    if P[k+1] == P[q]
        k = k+1
    π[q] = k
return π.
```

Running time:

Preprocessing time = $\Theta(m)$
Matching time = $\Theta(n)$.

Problem:

Compute the prefix fn π for the pattern ababbabbabbababbabb.

2. Apply KMP algorithm for the given text T and Pattern P.

(a) Text (T): B A B A B A B A B C A B A B C A B A B

Pattern (P): A B A B C A B A B.

(b) T: a b a b c a b c a b a b a b d

P: a b a b d.